Universidad Tecnológica Unidad Académica San Luis Río Colorado

RESULTADO DE APRENDIZAJE

Arquitectura en Capas y Patrones de Diseño

Capa de servicios y patrones Factory Method, Strategy y Facade
en la API del proyecto tienda_online (Unidad 1)

Desarrollo Web Integral

MAESTRO: JOSE PATIÑO

GRUPO: IDGS 8-1

ALUMNO: AARON ALBERTO FELIX GONZALEZ

San Luis Río Colorado, Sonora
25 de septiembre de 2026

Contenido

1. Que se hizo
2. Modelo de arquitectura del proyecto
3. Capa de servicios: services.py (Strategy)
4. Factory Method: crear_con_categoria_default en models.py
5. Facade: endpoint /precio-final/ en views.py y urls.py
6. Probar el endpoint
7. Pruebas unitarias nuevas
8. Resultado de python manage.py test
9. Tabla de patrones identificados

1. Que se hizo

Sobre el proyecto tienda_online de las Prácticas 2 a 5 (productos, categorías, autenticación JWT y pruebas unitarias) se agregó una capa de servicios en products/services.py para separar la lógica de negocio de las vistas, y se implementaron tres patrones de diseño: uno creacional (Factory Method), uno de comportamiento (Strategy) y uno estructural (Facade).

El resultado es un endpoint nuevo, /api/products/<id>/precio-final/, protegido con JWT, que devuelve el precio de un producto con el descuento que le corresponde según su categoría. Se agregaron dos pruebas unitarias para ese endpoint y todas las pruebas del proyecto pasan.

En este proyecto las rutas de la app están montadas en /api/ (tienda_online/urls.py tiene path('api/', include('products.urls'))), por eso el endpoint queda en /api/products/<id>/precio-final/ y no en /api/v1/. Las pruebas usan la misma ruta.

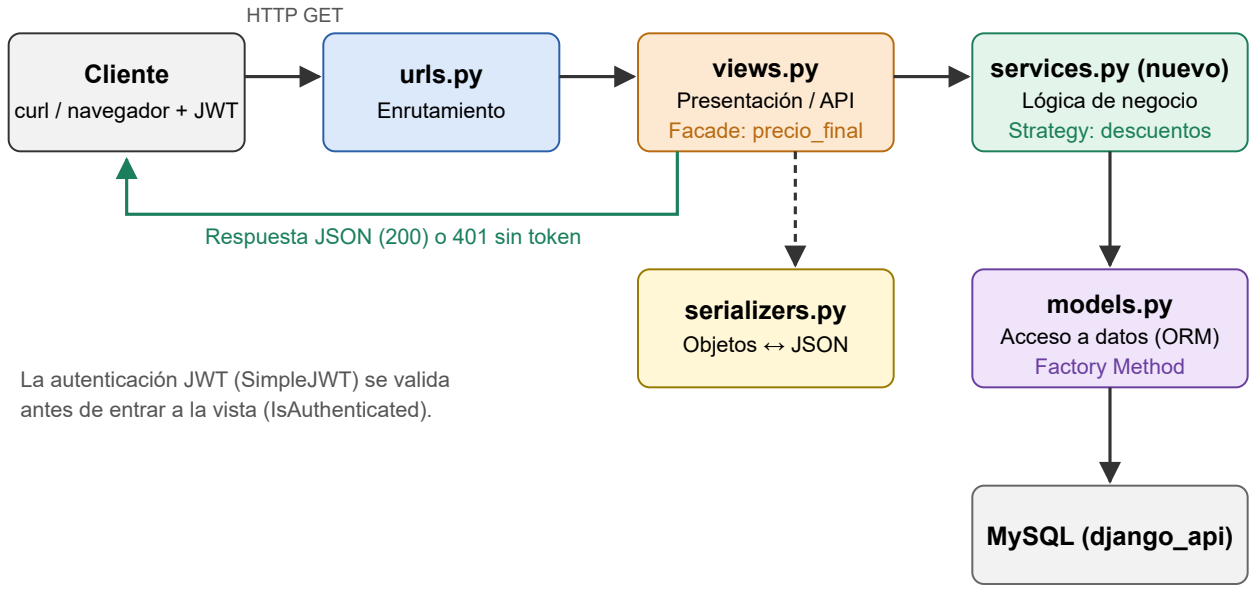
2. Modelo de arquitectura del proyecto

El proyecto sigue una arquitectura en capas basada en el patrón MVT (Model, View, Template) de Django. Como es una API REST con Django REST Framework, la parte de Template la cubren los serializers, que convierten los objetos a JSON. Con la capa nueva de servicios, cada capa tiene una sola responsabilidad:

urls.py: recibe la petición y la manda a la vista correcta.
views.py: capa de presentación; revisa la autenticación JWT y arma la respuesta.
services.py: capa de negocio; decide qué descuento aplicar.
models.py: capa de datos; define las entidades y accede a MySQL con el ORM.

Flujo de una petición: urls.py → views.py → services.py → models.py

Diagrama de la arquitectura en capas:



3. Capa de servicios: services.py (Strategy)

Se creó el archivo nuevo products/services.py. Tiene la clase base EstrategiaDescuento con el método calcular(), tres estrategias concretas (DescuentoElectronica con 10%, DescuentoRopa con 20% y SinDescuento) y la función obtener_estrategia(), que regresa la estrategia que corresponde al nombre de la categoría. Si la categoría no está en el diccionario, se usa SinDescuento.

products/services.py (archivo nuevo):

products/services.py

```
# Capa de servicios: aqui vive la logica de negocio, separada de las vistas.

# Patron de comportamiento: Strategy
# Cada estrategia encapsula una forma distinta de calcular el descuento
class EstrategiaDescuento:
    def calcular(self, precio):
        raise NotImplementedError


class DescuentoElectronica(EstrategiaDescuento):
    def calcular(self, precio):
        return precio * 0.90


class DescuentoRopa(EstrategiaDescuento):
    def calcular(self, precio):
        return precio * 0.80


class SinDescuento(EstrategiaDescuento):
    def calcular(self, precio):
        return precio


# Selecciona la estrategia segun el nombre de la categoria del producto
def obtener_estrategia(categoria_nombre):
    estrategias = {
        'Electronica': DescuentoElectronica(),
        'Ropa': DescuentoRopa(),
    }
    return estrategias.get(categoria_nombre, SinDescuento())
```

4. Factory Method: models.py

En el modelo Product se agregó el método de clase crear_con_categoria_default. Busca la categoría «Sin categoría» con get_or_create (si no existe la crea) y con ella crea el producto. Así, quien necesite un producto sin categoría no tiene que preocuparse por crearla. Con este método se creó el producto Mochila (id 6), que se usa en las pruebas del punto 6. Las líneas nuevas están resaltadas en verde.

products/models.py:

products/models.py

```
18  class Product(models.Model):
19      name = models.CharField(max_length=255, verbose_name='Nombre')
20      category = models.ForeignKey(
21          Category,
22          on_delete=models.CASCADE,
23          related_name='get_products',
24          verbose_name='Categoria',
25      )
26      description = models.TextField(verbose_name='Description')
27      price = models.DecimalField(max_digits=10, decimal_places=2,
    verbose_name='Price')
28      is_active = models.BooleanField(default=True)
29
30      class Meta:
31          verbose_name = 'Product'
32          verbose_name_plural = 'Products'
33          ordering = ['name']
34
35      def __str__(self):
36          return self.name
37
38      # Patron creacional: Factory Method
39      # Crea un producto asignando la categoria 'Sin categoría' cuando no se
    especifica una
40      @classmethod
41      def crear_con_categoria_default(cls, name, description, price):
42          categoria_default, _ = Category.objects.get_or_create(name='Sin categoría')
43          return cls.objects.create(
44              name=name,
45              description=description,
46              price=price,
47              category=categoria_default,
48          )
```

5. Facade: views.py y urls.py

En views.py se creó la vista precio_final. Es la fachada: con una sola llamada GET busca el producto, pide la estrategia a la capa de servicios y devuelve el precio original y el precio final. Está protegida con IsAuthenticated, así que sin token responde 401. Se importaron api_view, permission_classes, Response y obtener_estrategia. Las líneas nuevas están resaltadas en verde.

products/views.py:

products/views.py

```python
from rest_framework import viewsets
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from django_filters.rest_framework import DjangoFilterBackend
from .models import Category, Product
from .serializers import CategorySerializer, ProductSerializer
from .services import obtener_estrategia


class ProductViewSet(viewsets.ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
    filter_backends = [DjangoFilterBackend]
    filterset_fields = ['category']
    permission_classes = [IsAuthenticated]


class CategoryViewSet(viewsets.ModelViewSet):
    queryset = Category.objects.all()
    serializer_class = CategorySerializer
    permission_classes = [IsAuthenticated]


# Patron estructural: Facade
# Una sola llamada oculta la busqueda del producto, la eleccion de la
# estrategia de descuento y el calculo del precio final
@api_view(['GET'])
@permission_classes([IsAuthenticated])
def precio_final(request, pk):
    producto = Product.objects.get(pk=pk)
    estrategia = obtener_estrategia(producto.category.name)
    precio_con_descuento = estrategia.calcular(float(producto.price))
    return Response({
        'producto': producto.name,
        'precio_original': producto.price,
        'precio_final': round(precio_con_descuento, 2),
    })
```

En products/urls.py se importó la vista y se agregó la ruta nueva junto a las rutas del router.

products/urls.py:

products/urls.py

```
from django.urls import path, include
from .views import CategoryViewSet, ProductViewSet, precio_final
from rest_framework.routers import DefaultRouter

router = DefaultRouter()
router.register('products', ProductViewSet, basename='products')
router.register('categories', CategoryViewSet, basename='categories')

urlpatterns = [
    path('', include(router.urls)),
]

urlpatterns += [
    path('products/<int:pk>/precio-final/', precio_final),
]
```

6. Probar el endpoint

Con el servidor corriendo en http://127.0.0.1:8000 se llamó al endpoint con curl desde PowerShell, enviando el token access de usuario_jwt (guardado en la variable $token) en el header Authorization: Bearer. Se probaron tres productos de categorías distintas y el descuento cambia en cada uno. Al final se repitió la petición sin token y la API respondió 401.

Endpoint /precio-final/ con productos de distintas categorías:

Terminal — prueba del endpoint /precio-final/

```
PS C:\Users\aaron\Downloads\django-project> curl.exe -s -H "Authorization: Bearer $token"
http://127.0.0.1:8000/api/products/1/precio-final/
{"producto":"Laptop","precio_original":899.99,"precio_final":809.99}
PS C:\Users\aaron\Downloads\django-project> curl.exe -s -H "Authorization: Bearer $token"
http://127.0.0.1:8000/api/products/3/precio-final/
{"producto":"Camisa","precio_original":25.5,"precio_final":20.4}
PS C:\Users\aaron\Downloads\django-project> curl.exe -s -H "Authorization: Bearer $token"
http://127.0.0.1:8000/api/products/6/precio-final/
{"producto":"Mochila","precio_original":350.0,"precio_final":350.0}
PS C:\Users\aaron\Downloads\django-project> curl.exe -s -i
http://127.0.0.1:8000/api/products/1/precio-final/
HTTP/1.1 401 Unauthorized
{"detail":"Authentication credentials were not provided."}
```

Producto	Categoría	Estrategia	Precio original	Precio final
Laptop	Electronica	DescuentoElectronica (10%)	899.99	809.99
Camisa	Ropa	DescuentoRopa (20%)	25.50	20.40
Mochila	Sin categoría	SinDescuento	350.00	350.00

7. Pruebas unitarias nuevas

En products/tests.py se agregaron dos pruebas a la clase ProductTestCase de la Práctica 5. Esa clase ya autentica al usuario en setUp() con force_authenticate, por lo que la primera prueba espera 200 y que la respuesta traiga precio_final. La segunda quita la autenticación con force_authenticate(user=None) y espera 401. Las líneas nuevas están resaltadas en verde.

products/tests.py:

products/tests.py

```
81
82      def test_list_products_without_token(self):
83          self.client.force_authenticate(user=None)
84          response = self.client.get('/api/products/')
85          self.assertEqual(response.status_code, 401)
86
87      def test_precio_final_autenticado(self):
88          response = self.client.get(f'/api/products/{self.product.id}/precio-
    final/')
89          self.assertEqual(response.status_code, 200)
90          self.assertIn('precio_final', response.data)
91
92      def test_precio_final_sin_autenticar(self):
93          self.client.force_authenticate(user=None)
94          response = self.client.get(f'/api/products/{self.product.id}/precio-
    final/')
95          self.assertEqual(response.status_code, 401)
```

8. Resultado de python manage.py test

Se corrieron todas las pruebas del proyecto. Pasan las 8 de la Práctica 5 y las 2 nuevas, 10 en total. Los avisos W042 ya estaban desde las prácticas anteriores y no tienen que ver con este trabajo.

Ejecución de las pruebas:

Terminal — python manage.py test

```
PS C:\Users\aaron\Downloads\django-project> python manage.py test -v 2
Creating test database for alias 'default' ('test_django_api')...

Found 10 test(s).

WARNINGS:
products.Category: (models.W042) Auto-created primary key used when not defining a primary key
type, by default 'django.db.models.AutoField'.
        HINT: Configure the DEFAULT_AUTO_FIELD setting or the ProductsConfig.default_auto_field
attribute to point to a subclass of AutoField, e.g. 'django.db.models.BigAutoField'.
products.Product: (models.W042) Auto-created primary key used when not defining a primary key
type, by default 'django.db.models.AutoField'.
        HINT: Configure the DEFAULT_AUTO_FIELD setting or the ProductsConfig.default_auto_field
attribute to point to a subclass of AutoField, e.g. 'django.db.models.BigAutoField'.

System check identified 2 issues (0 silenced).
test_create_category (products.tests.CategoryTestCase.test_create_category) ... ok
test_list_categories (products.tests.CategoryTestCase.test_list_categories) ... ok
test_create_product (products.tests.ProductTestCase.test_create_product) ... ok
test_delete_product (products.tests.ProductTestCase.test_delete_product) ... ok
test_filter_products_by_category
(products.tests.ProductTestCase.test_filter_products_by_category) ... ok
test_list_products (products.tests.ProductTestCase.test_list_products) ... ok
test_list_products_without_token
(products.tests.ProductTestCase.test_list_products_without_token) ... ok
test_precio_final_autenticado (products.tests.ProductTestCase.test_precio_final_autenticado) ...
ok
test_precio_final_sin_autenticar
(products.tests.ProductTestCase.test_precio_final_sin_autenticar) ... ok
test_update_product (products.tests.ProductTestCase.test_update_product) ... ok

----------------------------------------------------------------------
Ran 10 tests in 7.242s

OK
Destroying test database for alias 'default' ('test_django_api')...

 OK
```

9. Tabla de patrones identificados

Patrón	Categoría	Evidencia (dónde se implementó)	Explicación
Factory Method	Creacional	products/models.py, método de clase Product.crear_con_categoria_default() (líneas 38 a 48).	Encapsula la creación de un Product en un método de clase: quien lo usa no construye la categoría, el método obtiene o crea «Sin categoría» y devuelve el producto listo. Es creacional porque controla cómo se crean los objetos.
Facade	Estructural	products/views.py, vista precio_final(), y su ruta en products/urls.py: /api/products/<id>/precio-final/.	Da una interfaz única y simple (un GET) que oculta varios pasos: buscar el producto, elegir la estrategia en services.py y calcular el precio. Es estructural porque organiza cómo se exponen varias partes del sistema detrás de una sola entrada.
Strategy	Comportamiento	products/services.py: EstrategiaDescuento, DescuentoElectronica, DescuentoRopa, SinDescuento y obtener_estrategia().	Define una familia de algoritmos de descuento intercambiables con la misma interfaz calcular(), y se elige uno en tiempo de ejecución según la categoría. Es de comportamiento porque cambia lo que hace el objeto sin cambiar a quien lo usa.